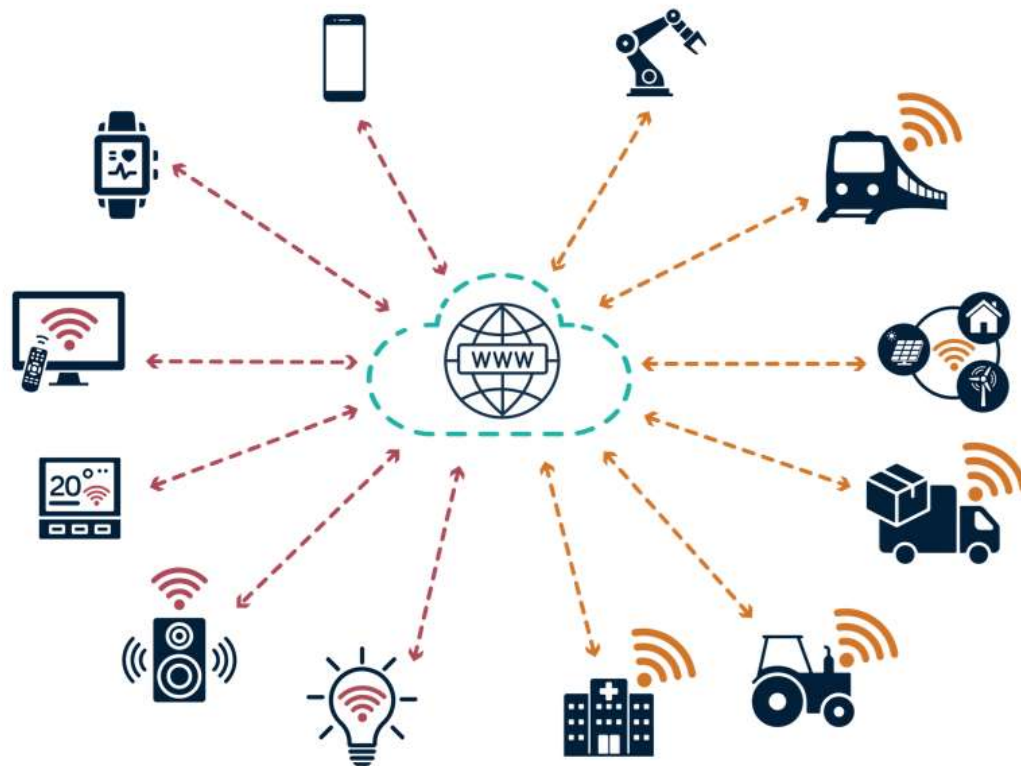


Ubiquitous Computing

COMP H4025

Lecturer: Simon McLoughlin

Lecture 4



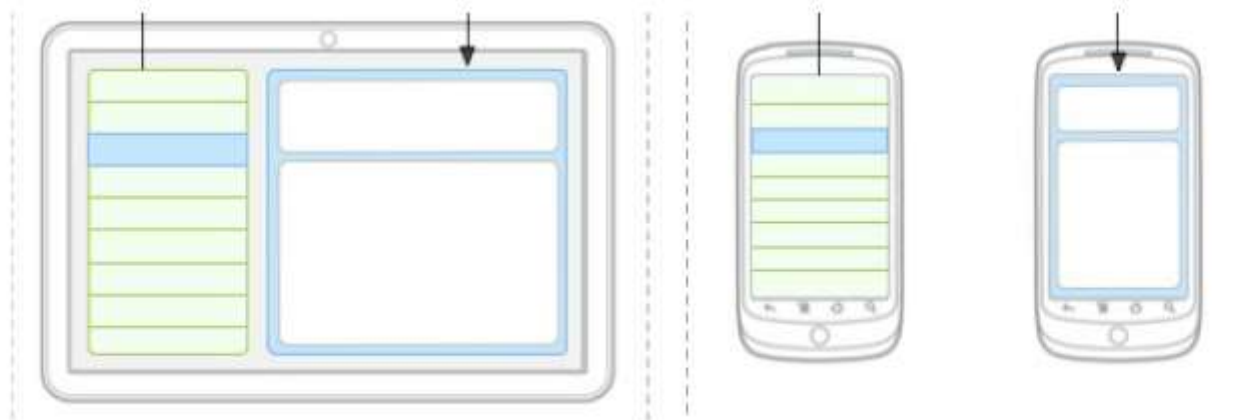
Lecture Overview

This week:

- Fragments
- Services
- Broadcast Receivers/Pending Intents
- Notifications

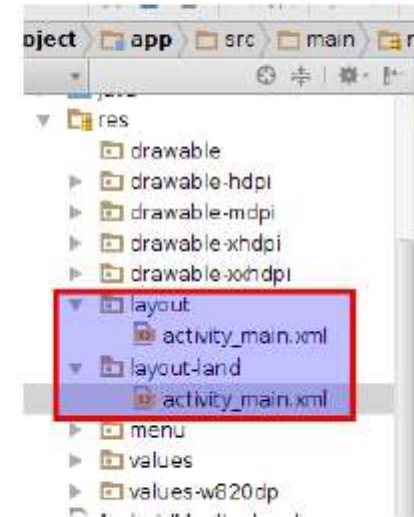
Layouts for different situations

- Your app can use different layout in different situations:
 - different device type (tablet vs phone vs watch)
 - different screen size
 - different orientation (portrait vs. landscape)
 - different country or locale (language, etc.)



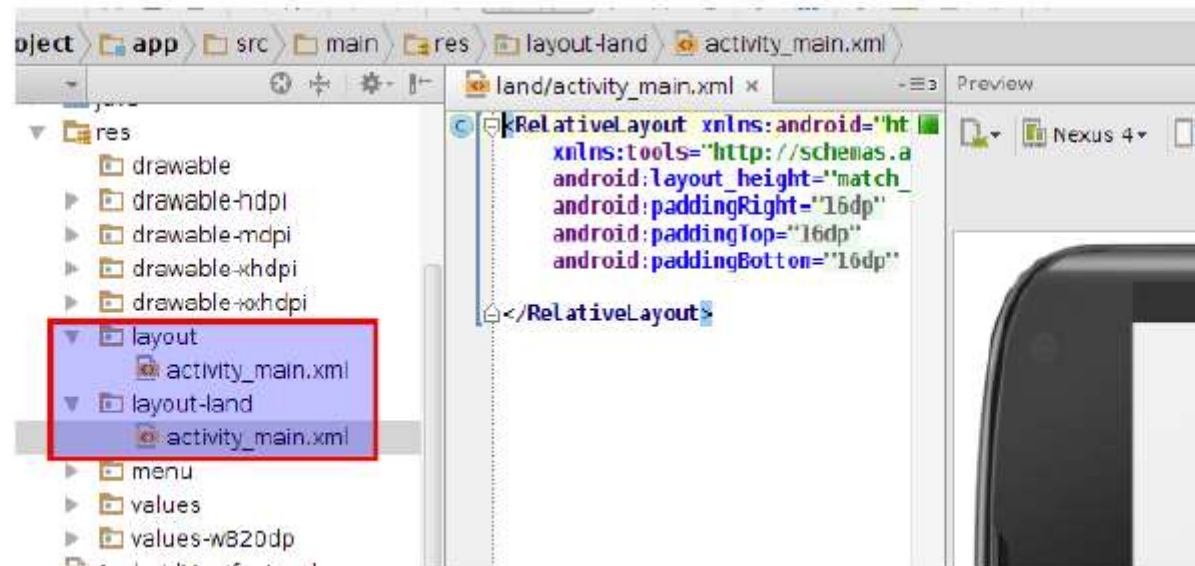
Layouts for different situations

- Your app will look for resource folder names with suffixes:
 - screen density (e.g. **drawable-hdpi**) ([link](#))
 - xhdpi: 2.0 (twice as many pixels/dots per inch)
 - hdpi: 1.5
 - mdpi: 1.0 (baseline)
 - ldpi: 0.75
 - screen size (e.g. **layout-large**) ([link](#))
 - small, normal, large, xlarge
 - orientation (e.g. **layout-land**)
 - portrait (), land (landscape)



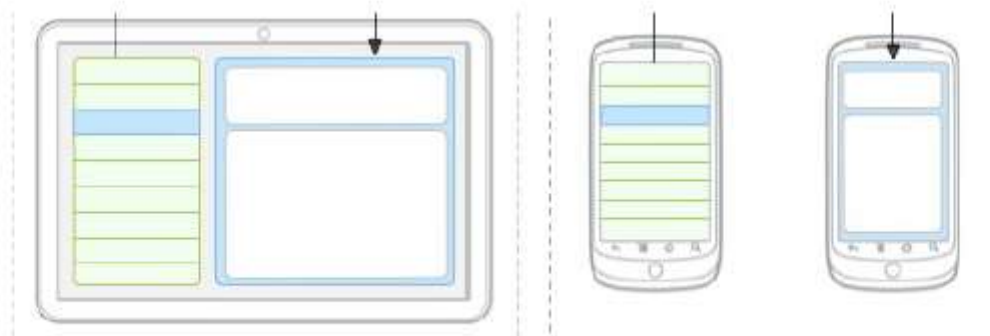
Portrait vs Landscape Layouts

- To create a different layout in landscape mode:
 - create a folder in your project called **res/layout-land**
 - place another copy of your activity's **layout XML file** there
 - modify it as needed to represent the differences



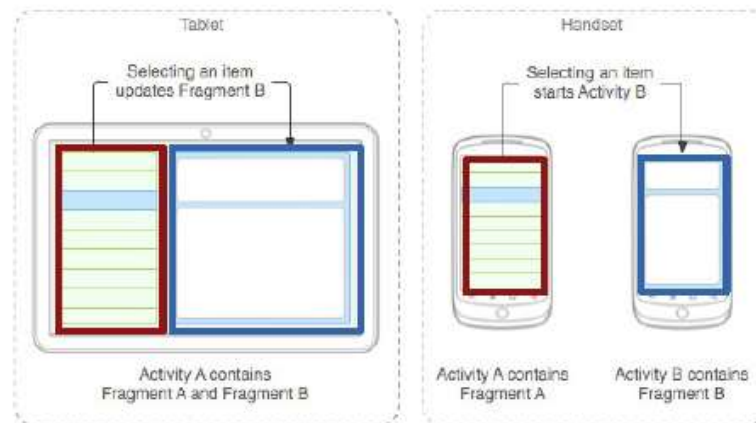
Dynamic Layouts for different situations

- You sometimes want your code to behave **situationally**.
 - In portrait mode, clicking a button should launch a new **activity**.
 - In landscape mode, clicking a button should launch a new **view**.



Fragments

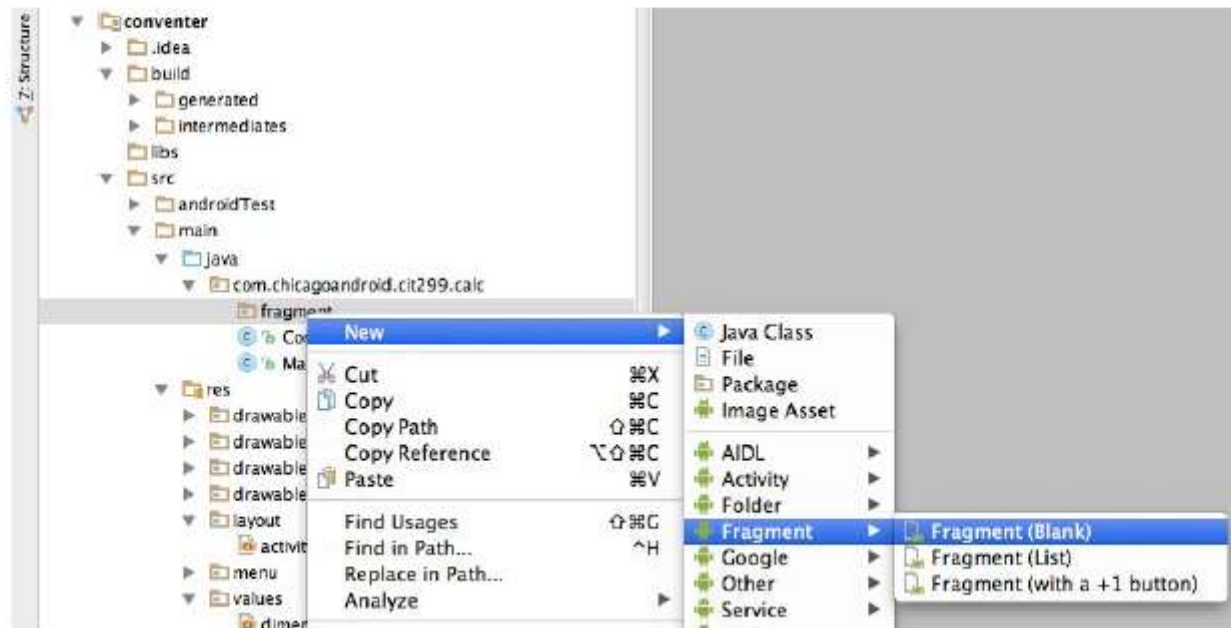
- **fragment:** A reusable segment of Android UI that can appear in an activity.
 - can help handle different devices and screen sizes
 - can reuse a common fragment across multiple activities
 - first added in Android 3.0 (*usable in older versions if necessary*)



- Fragments exist primarily to support more dynamic and flexible UI designs on large screens, such as tablets. Because a tablet's screen is much larger than that of a handset, there's more room to combine and interchange UI components.
- By dividing the layout of an activity into fragments, you become able to modify the activity's appearance at runtime.

Creating a Fragment

- In Android Studio, right-click app, click:
New → Fragment → Fragment (blank)
 - un-check boxes about "Include __ methods"
 - now create layout XML and Java event code as in an Activity



Using Fragments in Activity layout

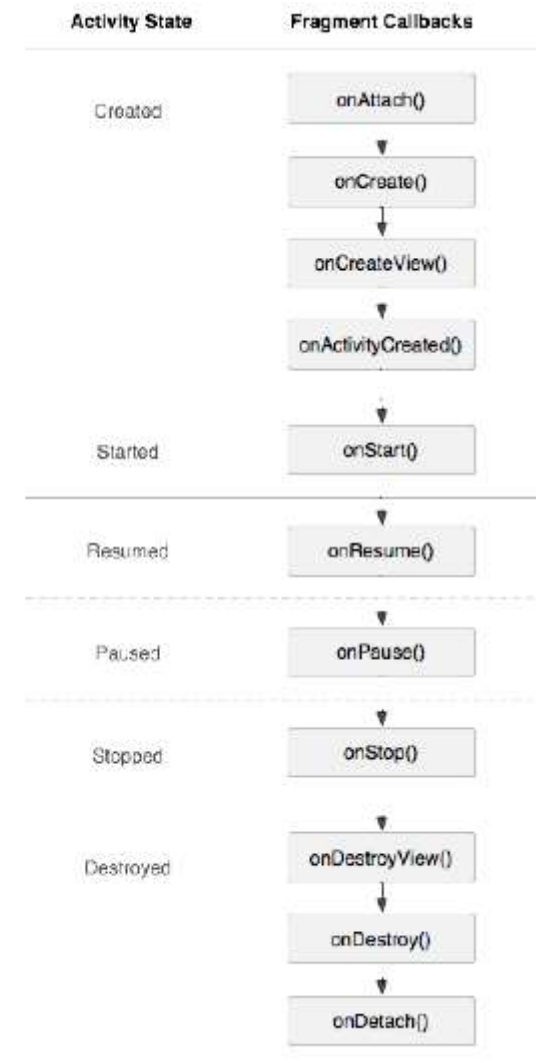
- Activity layout XML can include fragments.

```
<!-- activity_name.xml -->
<LinearLayout ...>
    <fragment ...
        android:id="@+id/id1"
        android:name="ClassName1"
        tools:layout="@layout/name1" />
    <fragment ...
        android:id="@+id/id2"
        android:name="ClassName2"
        tools:layout="@layout/name2" />
</LinearLayout>
```



Fragment lifecycle

- Fragments have a similar **life cycle** and events as activities.
- Important methods:
 - **onAttach** to glue fragment to its surrounding activity
 - **onCreate** when fragment is loading
 - **onCreateView** method that must return fragment's root UI view
 - **onActivityCreated** method that indicates the enclosing activity is ready
 - **onPause** when fragment is being left/exited
 - **onDetach** just as fragment is being deleted



Fragment vs Activity

- Fragment code is similar to activity code, with a few changes:
 - Many activity methods aren't present in the fragment, but you can call **getActivity** to access the activity the fragment is inside of.

```
Button b = (Button) findViewById(R.id.but);  
Button b = (Button) getActivity().findViewById(R.id.but);
```
 - Sometimes also use `getView` to refer to the activity's layout
 - Event handlers cannot be attached in the XML any more. :-(
 - Must be attached in Java code instead.
 - Passing information to a fragment (via Intents/Bundles) is trickier.
 - The fragment must ask its enclosing activity for the information.
 - Fragment initialization code must be mindful of order of execution.
 - Does it depend on the surrounding activity being loaded? Etc.
 - Typically move `onCreate` code to `onActivityCreated`.

Fragment onClickListener

- Activity:

```
<Button android:id="@+id/b1"
        android:onClick="onClickB1" ... />
```

- Fragment:

```
<Button android:id="@+id/b1" ... />
```

```
// in fragment's Java file
```

```
Button b = (Button) getActivity().findViewById(r.id.b1);
b.setOnClickListener(new View.OnClickListener() {
    @Override public void onClick(View view) {
        // whatever code would have been in onClickB1
    }
});
```

Activity with Intent Data

```
public class Name extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.name);

        // extract parameters passed to activity from intent
        Intent intent = getIntent();
        int name1 = intent.getIntExtra("id1", default);
        String name2 = intent.getStringExtra("id2", "default");

        // use parameters to set up the initial state
        ...
    }
    ...
}
```

Fragment with Intent Data

```
public class Name extends Fragment {
    @Override
    public View onCreateView(LayoutInflater inflater,
        ViewGroup container, Bundle savedInstanceState) {
        return inflater.inflate(R.layout.name, container, false);
    }

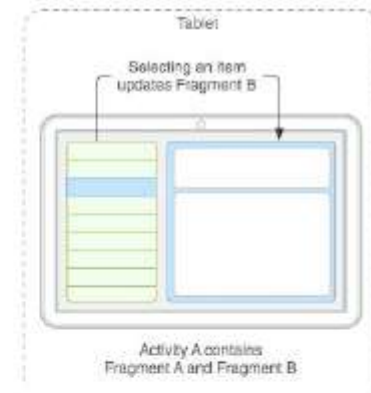
    @Override
    public void onActivityCreated(Bundle savedInstanceState) {
        super.onActivityCreated(savedInstanceState);

        // extract parameters passed to activity from intent
        Intent intent = getActivity().getIntent();
        int name1 = intent.getIntExtra("id1", default);
        String name2 = intent.getStringExtra("id2", "default");

        // use parameters to set up the initial state
        ...
    }
}
```

Communication between Fragments

- One activity might contain multiple fragments.
- The fragments may want to talk to each other.
 - Use activity's `getFragmentManager` method.
 - its `findFragmentById` method can access any fragment that has an id.



```
Activity act = getActivity();
if (act.getResources().getConfiguration().orientation ==
    Configuration.ORIENTATION_LANDSCAPE) {
    // update other fragment within this same activity
    FragmentClass fragment = (FragmentClass)
        act.getFragmentManager().findFragmentById(R.id.id);
    fragment.methodName(parameters);
}
```

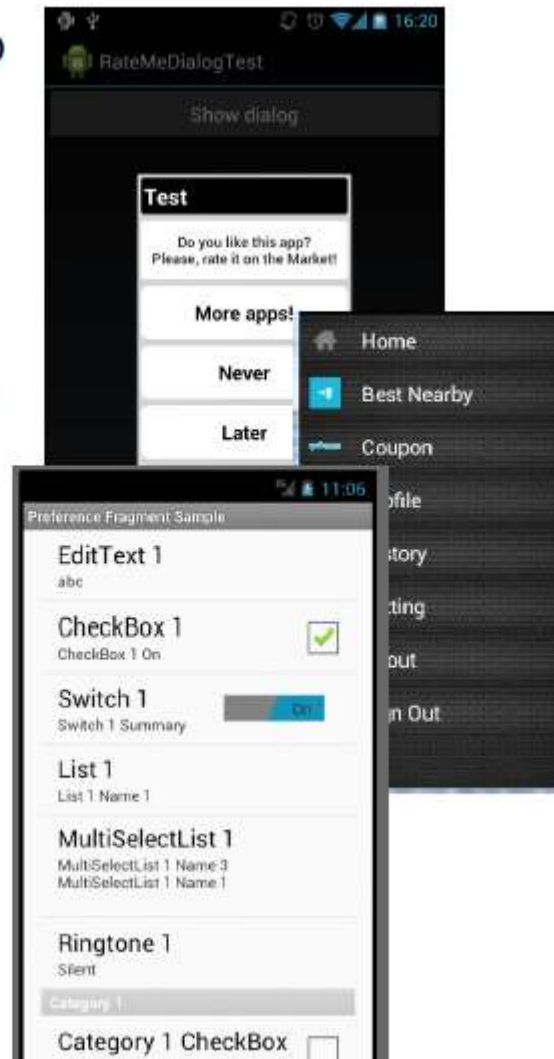
Dynamically switching/adding fragments

```
public void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    setContentView(R.layout.main);  
    OnClickListener listener = new OnClickListener() {  
        public void onClick(View view) {  
            Fragment fragment = null;  
            if(view == findViewById(R.id.button1)){  
                fragment = new FragmentOne();  
            } else {  
                fragment = new FragmentTwo();  
            }  
  
            FragmentManager manager = getSupportFragmentManager();  
            FragmentTransaction transaction = manager.beginTransaction();  
            transaction.replace(R.id.output, fragment);  
            transaction.commit();  
        }  
    };  
    Button btn1 = (Button)findViewById(R.id.button1);  
    btn1.setOnClickListener(listener);  
    Button btn2 = (Button)findViewById(R.id.button2);  
    btn2.setOnClickListener(listener);  
}
```

```
<?xml version="1.0" encoding="utf-8"?>  
<LinearLayout xmlns:android="http://schemas.android.c  
    xmlns:tools="http://schemas.android.com/tools"  
    android:layout_width="match_parent"  
    android:layout_height="match_parent"  
    android:orientation="vertical" >  
    <Button  
        android:id="@+id/button1"  
        android:layout_width="fill_parent"  
        android:layout_height="wrap_content"  
        android:text="@string/btn_label1"/>  
    <Button  
        android:id="@+id/button2"  
        android:layout_width="fill_parent"  
        android:layout_height="wrap_content"  
        android:text="@string/btn_label2"/>  
    <fragment  
        android:layout_width="match_parent"  
        android:layout_height="match_parent"  
        class="com.concretepage.android.FragmentOne"  
        android:id="@+id/output"/>  
</LinearLayout>
```

Fragment Subclasses

- `DialogFragment` - a fragment meant to be shown as a dialog box that pops up on top of the current activity.
- `ListFragment` - a fragment that shows a list of items as its main content.
- `PreferenceFragment` - a fragment whose main content is meant to allow the user to change settings for the app.

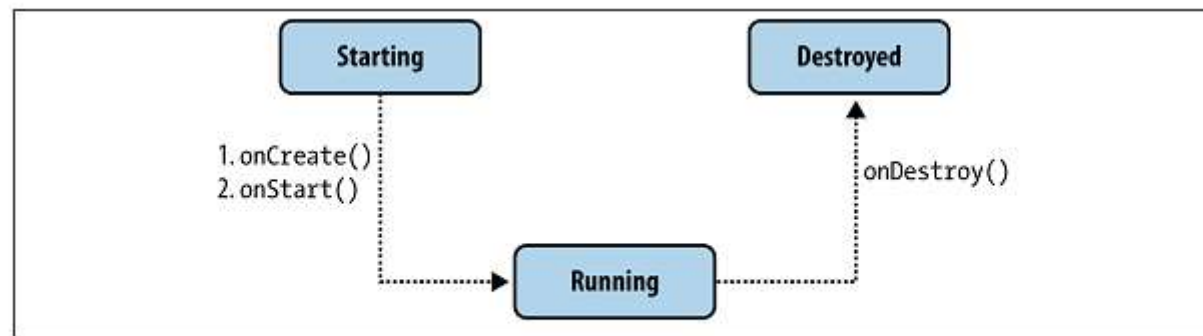


Services

- Services are among the main building blocks in Android. Unlike an activity, a service doesn't have a user interface; it is simply a piece of code that runs in the background of your application – but it is not a Thread!
- Services are used for processes that should run independently of activities, which may come and go. A twitter like application, for example, may need to create a service to periodically connect to the cloud and check for new statuses from the user's friends.
- Example: Google Play Music plays the music using a service.
- Example: Web browser runs a downloader service to retrieve a file
- The steps in creating a Service are:
 1. Create the Java class representing your service.
 2. Register the service in the AndroidManifest.xml file.
 3. Start the service.

Creating a Service

- The basic procedure for creating a service, as with activities and other main building blocks, is to subclass a Service class provided by the Android framework.

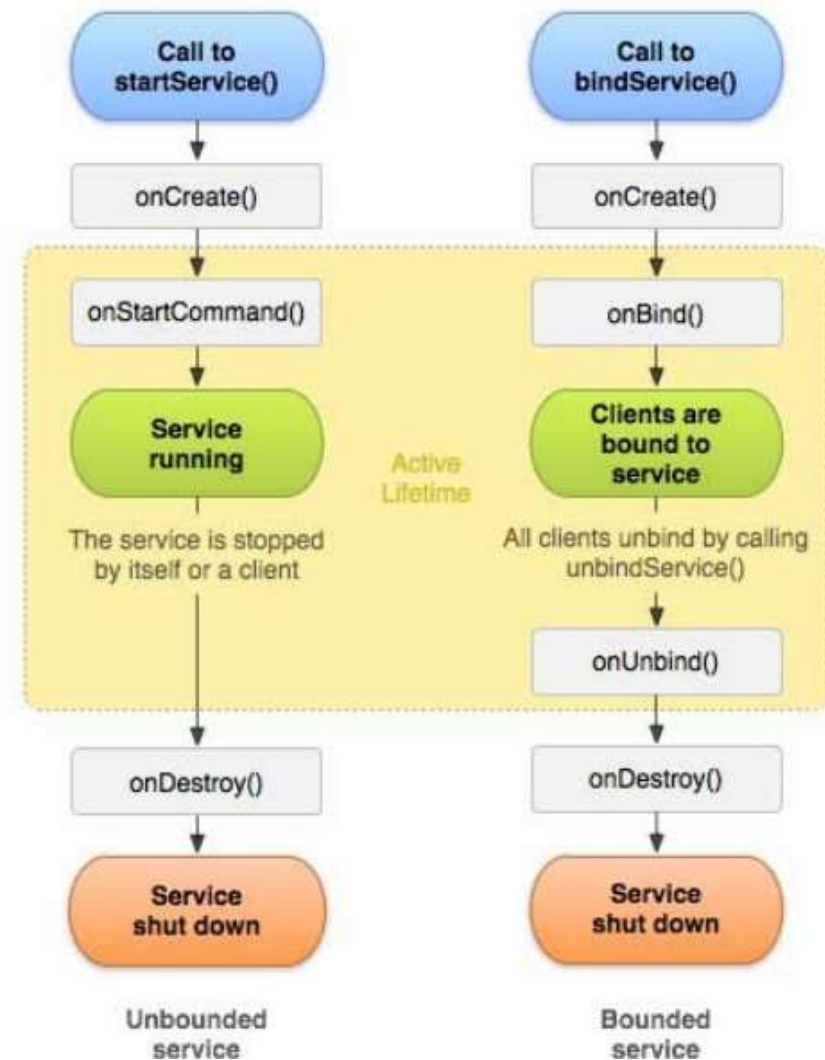


Service life cycle

- Next, we want to override some of the main life cycle methods, onCreate() is called when the service is created for the first time.
- onStartCommand() is called each time the service is started
- onDestroy() is called when the service is terminated
- A service can be bound or unbound. We'll focus on unbound services, where the life cycle of a service is not tied to the life cycle of the activities that started them.

Service Life Cycle

- A service is started by an app's activity using an intent.
- Service operation modes:
 - **start**: The service keeps running until it is manually stopped.
 - *we'll use this one*
 - **bind**: The service keeps running until no "bound" apps are left.
- Services have similar methods to activities for lifecycle events.
 - onCreate, onDestroy



Manifest Entry

- To allow your app to use the service, add the following to your app's `AndroidManifest.xml` configuration:

(Android Studio does this for you if you use the New Service option)

- the `exported` attribute signifies whether other apps are also allowed to use the service (`true=yes`, `false=no`)
- note that you must write a dot (`.`) before the class name below!

```
<application ...>
```

```
  <service
```

```
    android:name=".ServiceClassName"
```

```
    android:enabled="true"
```

```
    android:exported="false" />
```

Service Template

```
public class ServiceClassName extends Service {  
    /* this method handles a single incoming request */  
    @Override  
    public int onStartCommand(Intent intent, int flags, int id) {  
        // unpack any parameters that were passed to us  
        String value1 = intent.getStringExtra("key1");  
        String value2 = intent.getStringExtra("key2");  
  
        // do the work that the service needs to do ...  
  
        return START_STICKY;    // stay running  
    }  
  
    @Override  
    public IBinder onBind(Intent intent) {  
        return null;    // disable binding  
    }  
}
```


Finishing a Service

- When a service has completed a task, it can notify the app by "sending a broadcast" which the app can listen for:
 - As before, set an **action** in the intent to distinguish different kinds of results.

```
public class ServiceClassName extends Service {  
    @Override  
    public int onStartCommand(Intent tent, int flags, int id) {  
        // do the work that the service needs to do ...  
        ...  
        // broadcast that the work is done  
        Intent done = new Intent();  
        done.setAction("action");  
        done.putExtra("key1", value1); ...  
        sendBroadcast(done);  
  
        return START_STICKY;    // stay running  
    }  
}
```

Receiving a Broadcast

- Your activity can hear broadcasts using a BroadcastReceiver.
 - Extend BroadcastReceiver with the code to handle the message.
 - Any extra parameters in the message come from the service's intent.

```
public class ActivityClassName extends Activity {  
    ...  
  
    private class ReceiverClassName extends BroadcastReceiver {  
        @Override  
        public void onReceive(Context context, Intent intent) {  
            // handle the received broadcast message  
            ...  
        }  
    }  
}
```

Receiving a Broadcast

- Set up your activity to be notified when certain broadcast actions occur.
 - You must pass an **intent filter** specifying the action(s) of interest.

```
public class ActivityClassName extends Activity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        ...  
        IntentFilter filter = new IntentFilter();  
        filter.addAction("action");  
        registerReceiver(new ReceiverClassName(), filter);  
    }  
}
```

More on Broadcast Receivers later.....

IntentService

- It's important to note that your service is going to run on the main thread of the application, i.e., the UI thread.
- Because our service could be connecting to the cloud to pull down the latest data, we once again have the problem of networking on the UI thread.
- Android SDK provides a subclass of service called `IntentService` that is similar to a regular service, with two main exceptions: whatever work has to be done in `onHandleIntent()` will execute on a separate worker thread, and once it's done, the service will stop.

JobService and JobScheduler

- Because of recent restrictions on Android to improve battery life, all background work including periodic tasks is recommended to be scheduled through the JobScheduler.
- The job schedule is part of the android platform and maintains background jobs of all applications installed on the device in one place allowing for optimal usage of device resources. JobScheduler can be used to run short and long running tasks such as downloading data, processing of data, etc.
- The framework will be intelligent about when it executes jobs, and attempt to batch and defer them as much as possible. Typically if you don't specify a deadline on a job, it can be run at any moment depending on the current state of the JobScheduler's internal queue.
- You construct JobInfo objects and pass them to the JobScheduler with `schedule(android.app.job.JobInfo)`. When the criteria declared are met, the system will execute this job on your application's JobService. You identify the service component that implements the logic for your job when you construct the JobInfo using `JobInfo.Builder.Builder(int, android.content.ComponentName)`.

Broadcast Receivers

- A broadcast receiver is an Android component which allows you to **register for system or application events or 'broadcasts'**. All registered receivers for an event are notified by the Android runtime once this event happens.
- For example, applications can register for the **ACTION_BATTERY_LOW** system event which is fired once the Android system detects the battery has dropped beyond a certain level
- A receiver can be **registered via the AndroidManifest.xml** file or registered dynamically via the Context.registerReceiver() method.
- The implementing class for a receiver **extends the BroadcastReceiver** class and if the event for which the broadcast receiver has registered happens, the **onReceive() method** of the receiver is called by the Android system.
- The Android system excludes all receivers from receiving intents by default if the corresponding application has **never been started** by the user or if the user **explicitly stopped** the application via the Android menu.

Broadcast Receiver for Phone State

- Here is an example that defines a broadcast receiver which listens to telephone state changes. If the phone receives a phone call, then the receiver will be notified and log a message.

```
<uses-sdk android:minsdkversion="15" />

<uses-permission android:name="android.permission.READ_PHONE_STATE" >
</uses-permission>

<application
    android:icon="@drawable/icon"
    android:label="@string/app_name" >
    <activity
        android:name=".MainActivity"
        android:label="@string/title_activity_main" >
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <receiver android:name="MyPhoneReceiver" >
        <intent-filter>
            <action android:name="android.intent.action.PHONE_STATE" >
            </action>
        </intent-filter>
    </receiver>
</application>
```

Broadcast Receiver for Phone State

- Here is the receiver class that will be launched in reaction to a phone call:

```
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.os.Bundle;
import android.telephony.TelephonyManager;
import android.util.Log;

public class MyPhoneReceiver extends BroadcastReceiver {

    @Override
    public void onReceive(Context context, Intent intent) {
        Bundle extras = intent.getExtras();
        if (extras != null) {
            String state = extras.getString(TelephonyManager.EXTRA_STATE);
            Log.w("MY_DEBUG_TAG", state);
            if (state.equals(TelephonyManager.EXTRA_STATE_RINGING)) {
                String phoneNumber = extras
                    .getString(TelephonyManager.EXTRA_INCOMING_NUMBER);
                Log.w("MY_DEBUG_TAG", phoneNumber);
            }
        }
    }
}
```

Sending a Broadcast from your App

- If you want your app to send a system wide broadcast you can do it from any component that has application context, e.g. an activity.
- Simply call the `sendBroadcast` method from the activity and pass in an `Intent` with an action that defines the type of broadcast, e.g. fr

```
Intent intent = new Intent();  
intent.setAction("com.journaldev.CUSTOM_INTENT");  
sendBroadcast(intent);
```

Pending Intents

- If you want some action to take place at **some point in the future** in response to some event you should use a PendingIntent instead of an Intent object.
- For example, lets say you want to set an alert or an alarm in your activity, when the alarm goes off you want an Intent to be started and a Broadcast Receiver to then react to this Intent. As you do not want the Intent to be started straight away but rather when the Alarm goes off you should use a PendingIntent.
- In the example of an Activity that wants to invoke some action using an Intent in response to some event, the activity **might not be around** to launch the Intent when the event occurs so it should specify a Pending Intent, so even if the Activity process is stopped the component can still be launched by the Pending Intent.
- When you start or pass a PendingIntent to another Android component, that component performs the action described by the Intent with the **same set of permissions** as the component that created the Intent.

BroadcastReceiver and Pending Intents

- Here is an example to schedule a receiver via the Android alarm manager system service. The receiver will vibrate the phone and display a toast when it is called.
- We must register the receiver in manifest as before and give the app permission to vibrate the device

```
<uses-sdk android:minsdkversion="15" />

<uses-permission android:name="android.permission.VIBRATE" >
</uses-permission>

<application
    android:icon="@drawable/icon"
    android:label="@string/app_name" >
    <activity
        android:name=".AlarmActivity"
        android:label="@string/app_name" >
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />

            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <receiver android:name="MyBroadcastReceiver" >
    </receiver>
</application>
```

BroadcastReceiver class

- Here is the broadcast receiver class that displays the Toast and vibrates the phone

```
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.os.Vibrator;
import android.widget.Toast;

public class MyBroadcastReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "Don't panik but your time is up!!!!.",
            Toast.LENGTH_LONG).show();
        // vibrate the mobile phone
        Vibrator vibrator = (Vibrator) context.getSystemService
        (Context.VIBRATOR_SERVICE);
        vibrator.vibrate(2000);
    }
}
```

Activity to Set Alarm and define PendingIntent

```
import android.app.Activity;
import android.app.AlarmManager;
import android.app.PendingIntent;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;
```

```
public class AlarmActivity extends Activity {
```

```
    /** Called when the activity is first created. */
```

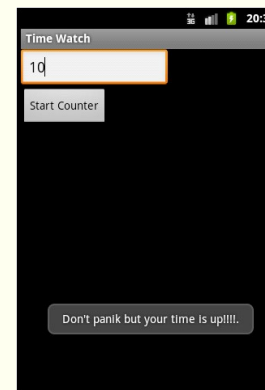
```
    @Override
```

```
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
    }
```

```
    public void startAlert(View view) {
        EditText text = (EditText) findViewById(R.id.time);
        int i = Integer.parseInt(text.getText().toString());
        Intent intent = new Intent(this, MyBroadcastReceiver.class);
        PendingIntent pendingIntent = PendingIntent.getBroadcast
        (this, 234324243, intent, 0);
        AlarmManager alarmManager = (AlarmManager) getSystemService(ALARM_SERVICE);
        alarmManager.set(AlarmManager.RTC_WAKEUP, System.currentTimeMillis()
            + (i * 1000), pendingIntent);
        Toast.makeText(this, "Alarm set in " + i + " seconds",
            Toast.LENGTH_LONG).show();
    }
}
```

```
// Set the alarm to start at approximately 2:00 p.m.
Calendar calendar = Calendar.getInstance();
calendar.setTimeInMillis(System.currentTimeMillis());
calendar.set(Calendar.HOUR_OF_DAY, 14);

// With setInexactRepeating(), you have to use one of the AlarmManager interval
// constants--in this case, AlarmManager.INTERVAL_DAY.
alarmMgr.setInexactRepeating(AlarmManager.RTC_WAKEUP, calendar.getTimeInMillis(),
    AlarmManager.INTERVAL_DAY, alarmIntent);
```

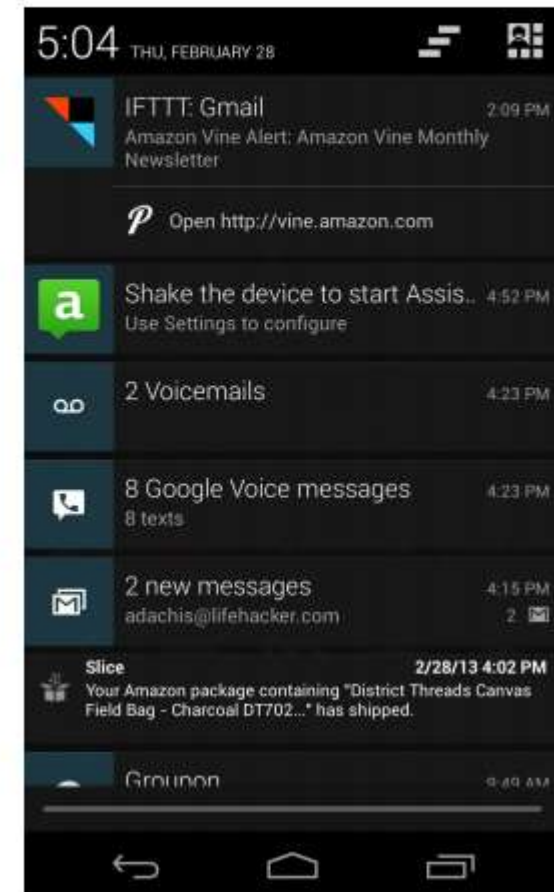


- Note here that the PendingIntent is going to perform a Broadcast as can be seen by PendingIntent.getBroadcast(..) but other options such as PendingIntent.getActivity() are also available to use a pending intent to start a new activity.

- Note also that a PendingIntent requires an Intent object but not vice versa

Notifications

- A notification is a message you can display to the user outside of your application's normal UI. When you tell the system to issue a notification, it first appears as an icon in the **notification area**.
- To see the details of the notification, the user opens the **notification drawer**. Both the notification area and the notification drawer are system-controlled areas that the user can view at any time.
- A Notification object must contain the following:
 - A small icon, set by `setSmallIcon()`
 - A title, set by `setContentTitle()`
 - Detail text, set by `setContentText()`



Creating Notifications in Android

- Create a notification using a `NotificationBuilder`.
- Use `NotificationManager` to send out the notification.

```
Notification.Builder builder = new Notification.Builder(this)
    .setContentTitle("title")
    .setContentText("text")
    .setAutoCancel(true)
    .setSmallIcon(R.drawable.icon);
Notification notification = builder.build();

NotificationManager manager = (NotificationManager)
    getSystemService(Context.NOTIFICATION_SERVICE);
manager.notify(ID, notification);
```

- The `Notification.Builder` provides a builder interface to create an `Notification` object. You use a `PendingIntent` to specify the action which should be performed once the user selects the notification.
- The `Notification.Builder` allows you to add up to three buttons with definable actions for the notification.

Creating Notifications in Android

```
import android.app.Activity;
import android.app.Notification;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;

public class CreateNotificationActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
    }

    public void createNotification(View view) {
        // Prepare intent which is triggered if the
        // notification is selected
        Intent intent = new Intent(this, NotificationReceiverActivity.class);
        PendingIntent pIntent = PendingIntent.getActivity(this, 0, intent, 0);

        // Build notification
        // Actions are just fake
        Notification noti = new Notification.Builder(this)
            .setContentTitle("New mail from " + "test@gmail.com")
            .setContentText("Subject").setSmallIcon(R.drawable.icon)
            .setContentIntent(pIntent)
            .addAction(R.drawable.icon, "Call", pIntent)
            .addAction(R.drawable.icon, "More", pIntent)
            .addAction(R.drawable.icon, "And more", pIntent).build();
        NotificationManager notificationManager = (NotificationManager)
            getSystemService(NOTIFICATION_SERVICE);
        // hide the notification after its selected
        noti.flags |= Notification.FLAG_AUTO_CANCEL;

        notificationManager.notify(0, noti);
    }
}
```

Creating Notifications in Android

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >

    <Button
        android:id="@+id/button1"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:onClick="createNotification"
        android:text="Create Notification" >
    </Button>

</LinearLayout>
```

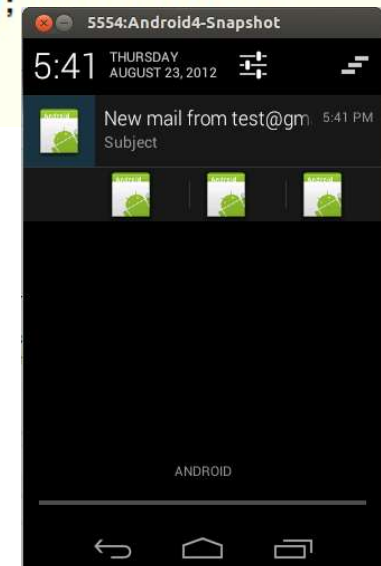
```
import android.app.Activity;
import android.os.Bundle;
```

```
public class NotificationReceiverActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.result);
    }
}
```

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >

    <TextView
        android:id="@+id/textview1"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="This is the result activity opened from the notification" >
    </TextView>

</LinearLayout>
```



Notification Channels

- Starting in Android 8.0 (API level 26), all notifications must be assigned to a channel. For each channel, you can set the visual and auditory behavior that is applied to all notifications in that channel. Then, users can change these settings and decide which notification channels from your app should be intrusive or visible at all.

```
if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.O) {  
    NotificationChannel channel = new NotificationChannel( id: "10002",  
        name: "CHANNEL_NAME",  
        NotificationManager.IMPORTANCE_DEFAULT);  
    channel.setDescription("CHANNEL_DESCRIPTION");  
    notificationManager.createNotificationChannel(channel);  
}  
  
// Prepare intent which is triggered if the  
// notification is selected  
Intent intent = new Intent( packageContext: this, NotificationOpenActivity.class);  
PendingIntent pIntent = PendingIntent.getActivity( context: this, (int) System.currentTimeMillis()  
  
// Build notification  
// Actions are just fake  
NotificationCompat.Builder noti = new NotificationCompat.Builder( context: this, channelId: "10002")
```

